

Answer Key Evaluation DEP

Expanded architecture: Frontend, Backend, Database, OCR, External Services

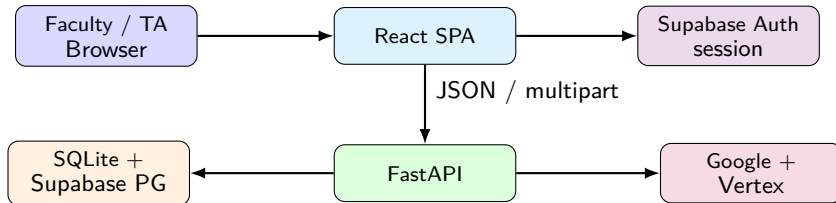
Project team

May 12, 2026

Deck roadmap (five pillars + extrapolation)

- 1 **Frontend** — React (Vite), routing, auth context, service layer calling FastAPI, Evaluate UX.
- 2 **Backend** — FastAPI routers, service orchestration, batch/ZIP/Drive pipelines, code-eval bridge.
- 3 **Database** — SQLite (`evaluation.db`), pooled WAL mode, Supabase Postgres (`result_cache`, `pdf_page_map`), cache key evolution.
- 4 **OCR** — Legacy OCRService, throughput-oriented OptimizedOCRService, dual-pass Pro path in optimized batch, preprocessing.
- 5 **External services** — Google Drive/Sheets APIs, Vertex / Gemini, Supabase Auth + DB URL, env configuration matrix.
- 6 **Extrapolation** — Scaling, observability, product surface (rubrics, review queues), security hardening.

End-to-end data flow (conceptual)



Typical grading loop: configure answer key in UI → trigger ZIP/Drive/PDF batch → backend downloads bytes → OCR extracts structured answers → scorer compares to key → results persisted + optional Sheets export + optional Drive rename.

Repository roots (top-level layout)

AnswerKeyEvaluationDEP/

- `frontend-new-design/` — Vite + React; browser talks to Supabase (auth/data) + FastAPI (`VITE_BACKEND_URL`).
- `backend/` — `main.py`, `api/`, `services/`, local `evaluation.db`, optional `vertex_key.json` / `sa-keys/`.
- `backend/migrations/` — SQL for Supabase (`result_cache`, `pdf_page_map`, cache scope columns).
- `c_code_evaluator/` — image→C extraction, compile-fix, LeetCode-style runners (`evaluate_c_from_image.py`, `leetcode_mode.py`).
- Root `README.md`, `IMPLEMENTATION_SUMMARY.md` — how to run FE+BE together.

File mapping — frontend src/ entry & routing

Path	Role
index.jsx	Mounts React root; imports Tailwind/global CSS.
App.jsx	Wraps AuthProvider around Routes.
Routes.jsx	BrowserRouter: public /login-register; protected dashboards, /evaluate/:evaluationId.
components/ProtectedRoute.jsx	If no Supabase session, Navigate to login; loading spinner while auth resolves.
components/ErrorBoundary.jsx	Catches render errors so one bad page does not white-screen the SPA.
components/ScrollToTop.jsx	Scroll reset on route changes.
components/ui/*	Shared primitives: Header, Button, Input, Select, modals, breadcrumbs.
utils/cn.js	Tailwind class-name helper (conditional classes).

File mapping — frontend services & context

Path	Role
context/AuthContext.jsx	Subscribes to Supabase auth; loads profiles row (retries + metadata fallback).
services/supabaseClient.js	createClient; project-scoped storageKey; stale-token cleanup.
services/authService.js	signIn, signUp, inserts profiles, user-facing error messages.
services/backendService.js	fetch to FastAPI /api/*; requireEvaluationId; ZIP multipart uploads.
services/evaluationService.js	Supabase evaluations, duties, answer-key persistence for UI state.
services/resultsService.js	Fetches/polls aggregated grading results for display (wrappers around Supabase/API).
services/courseService.js	Course CRUD / listing from Supabase.
services/studentService.js	Student roster helpers where used by UI.
services/teamService.js, invitationService.js	Team/invite flows on faculty pages.

File mapping — frontend pages (product surfaces)

Path	Role
pages/login-register/*	Login/register forms + optional social auth shell.
pages/faculty-dashboard/*	Landing for professors; CourseCard navigation.
pages/course-detail/index.jsx	Single course; links into evaluations.
pages/evaluate/index.jsx	Core grading UI : answer key sources, ZIP/Drive/PDF triggers, live progress.
pages/dashboard-overview/*	Sprint/progress widgets for ops-style dashboard.
pages/kanban-board/*	Board columns, filters, cards.
pages/task-detail/*	Rich task view: markdown, attachments, comments, activity.
pages/sprint-planning/*	Backlog, velocity chart, capacity.
pages/analytics-dashboard/index.jsx	Metrics dashboard page.
pages/team-management/*	Workspace settings, invites, members table.

Part I

Frontend stack & UX surfaces

Frontend stack

- **Runtime:** React with **Vite** (frontend-new-design/).
- **Routing:** react-router-dom — public login vs protected app shell.
- **Auth:** Supabase JS client; AuthContext wraps profile load + session; ProtectedRoute gates pages.
- **HTTP to FastAPI:** backendService.js — base URL VITE_BACKEND_URL, JSON helpers and multipart ZIP uploads.
- **Product UI:** Faculty dashboard, course detail, **Evaluate** (per :evaluationId), Kanban, task detail, analytics — aligns with academic ops workflows beyond raw OCR.

Routes (application map)

frontend-new-design/src/Routes.jsx

```
import EvaluatePage from "../pages/evaluate";

<RouterRoutes>
  <Route path="/login-register" element={<LoginRegister />} />
  <Route path="/dashboard-overview" element={<ProtectedRoute><DashboardOverview /></ProtectedRoute>} />
  <Route path="/faculty-dashboard" element={<ProtectedRoute><FacultyDashboard /></ProtectedRoute>} />
  <Route path="/course/:courseId" element={<ProtectedRoute><CourseDetail /></ProtectedRoute>} />
  <Route path="/evaluate/:evaluationId" element={<ProtectedRoute><EvaluatePage /></ProtectedRoute>} />
  <Route path="/kanban-board" element={<ProtectedRoute><KanbanBoard /></ProtectedRoute>} />
  <Route path="/analytics-dashboard" element={<ProtectedRoute><AnalyticsDashboard /></ProtectedRoute>} />
</RouterRoutes>
```

Supabase client (frontend only)

Auth project isolation + storage key (supabaseClient.js).

```
import { createClient } from '@supabase/supabase-js';

const supabaseUrl = import.meta.env.VITE_SUPABASE_URL;
const supabaseAnonKey = import.meta.env.VITE_SUPABASE_ANON_KEY;

const PROJECT_REF = supabaseUrl?.match(/https:\/\/(.+)\.supabase\.co\/)?.[1];
const authOptions = { lock: false, debug: true };
if (PROJECT_REF) {
  authOptions.storageKey = `sb-${PROJECT_REF}-auth-token`;
}

export const supabase = createClient(supabaseUrl, supabaseAnonKey, {
  auth: authOptions,
});
```

Backend calls + evaluation scoping guard

Every batch call must carry a real `evaluationId` so cache rows never leak across quizzes (`backendService.js`).

```
const BACKEND_URL = import.meta.env.VITE_BACKEND_URL || 'http://localhost:8000';

const requireEvaluationId = (evaluationId, action) => {
  if (!evaluationId || String(evaluationId).trim().toLowerCase() === 'default') {
    throw new Error(`Missing evaluationId for ${action}. Every processing run must be tied to a specific quiz
      ...`);
  }
  return String(evaluationId).trim();
};

async processDriveFolder(driveFolderUrl, evaluationId, forceReprocess, groupMultiplePages) {
  const evalId = requireEvaluationId(evaluationId, 'processDriveFolder');
  return api('/batch/process-folder-optimized', {
    method: 'POST',
    body: JSON.stringify({
      folder_url: driveFolderUrl,
      evaluation_id: evalId,
      rename_drive_inline: true,
      group_multiple_pages: groupMultiplePages,
    }),
  });
}
```

Evaluate page: answer key ingestion pattern

Same page composes Drive extract, manual JSON, and upload flows (evaluate/index.jsx).

```
1 const loadFromDrive = async () => {  
2   setLoading(true); setErr('');  
3   try {  
4     const res = await backendService.extractAnswerKeyFromDrive(driveUrl);  
5     await evaluationService.saveAnswerKey(evaluation.id, res.answer_key);  
6     onKeyLoaded(res.answer_key);  
7   } catch (e) { setErr(e.message); }  
8   finally { setLoading(false); }  
9 };
```

Related services: evaluationService.js, resultsService.js, templates via /download-answer-sheet-template.

Bootstrap: index.jsx + App.jsx

```
// index.jsx
import { createRoot } from "react-dom/client";
import App from "./App";
root.render(<App />);

// App.jsx
import { AuthProvider } from "../context/AuthContext";
function App() {
  return (
    <AuthProvider>
      <Routes />
    </AuthProvider>
  );
}
```

ProtectedRoute.jsx (session gate)

```
1 const ProtectedRoute = ({ children }) => {  
2   const { user, loading, profile } = useAuth();  
3   if (loading) {  
4     return ( /* full-screen spinner */ null );  
5   }  
6   if (!user) {  
7     return <Navigate to="/login-register" replace />;  
8   }  
9   return children;  
0 };
```

profile is loaded in AuthContext from Supabase profiles with retries.

evaluationService.js: Supabase evaluations

```
1 async getEvaluationById(evaluationId) {  
2   const { data, error } = await supabase  
3     .from('evaluations')  
4     .select(`  
5       *,  
6       courses(id, title, code, master_sheet_url, instructor_id),  
7       evaluation_duties(assignee_id, profiles(id, name, role))  
8     `)  
9     .eq('id', evaluationId)  
10    .single();  
11    if (error) throw error;  
12    return data;  
13  }
```

authService.js: sign-in wrapper

```
1 async signIn(email, password) {  
2   const { data, error } = await supabase.auth.signInWithPassword({  
3     email, password  
4   });  
5   if (error) {  
6     if (error.message === 'Invalid login credentials') {  
7       error.message = 'Invalid email or password...';  
8     }  
9     throw error;  
10  }  
11  return data;  
12 }
```

Frontend extrapolation (future)

- Unified **API client** layer with typed OpenAPI-generated stubs (reduce stringly-typed paths).
- **Streaming UI** for long batch jobs (SSE or WebSocket) instead of polling-only patterns where applicable.
- **Role-aware navigation** (professor vs TA vs admin) driven from Supabase profiles.
- Offline-safe **draft evaluations** and optimistic saves for answer-key JSON.
- Accessibility pass on Evaluate (keyboard, contrast, live regions for progress).

Browser: /evaluate/:evaluationId — answer key panel + batch controls

Part II

FastAPI backend & service layer

Application composition

backend/main.py: lifespan hooks warm Supabase pool; mount three routers.

```
app = FastAPI(title="Automated Answer Sheet Evaluation System", lifespan=lifespan)

app.include_router(endpoints.router)
app.include_router(batch_endpoints_optimized.router)
app.include_router(code_eval_endpoints.router)
```

CORS: CORS_ORIGINS comma-list enables credentials; wildcard fallback avoids cookie assumptions (Supabase bearer from FE).

/api/* standard router (selected)

backend/api/endpoints.py exposes the “classic” pipeline and tooling:

- GET /status, /performance
- /answer-key, extract/upload/set-manual
- POST /process-zip, /process-zip/start
- POST /process-drive-folder
- POST /export-to-sheets,
- /export-student-responses
- POST /full-pipeline, sync/rename variants
- /results, /cache/*, /database/optimize
- /pipeline-runs/{run_id}/*
- /rename-drive-files*

Optimized batch + PDF runs (/api/batch/*, /api/pdf-runs/*)

- POST /batch/process-folder-optimized (+ /start) — main Drive grading throughput path.
- GET /batch/processing-status/{id}, /batch/ocr-debug/{id}, problem-file downloads.
- DELETE /cache/purge-evaluation/{evaluation_id} — tenant-safe cache purge.
- POST /batch/process-pdf-optimized* — PDF upload pipeline.
- GET /pdf-runs/{id}/mapping* — page ↔ student reconciliation and CSV export.
- POST /pdf-runs/{id}/reconcile, image endpoints for QA.

Code evaluation router (/api/code-eval/*)

Bridges handwritten/uploaded C submissions with `c_code_evaluator` (OCR → compile-fix → LeetCode-style cases).

```
router = APIRouter(prefix="/api/code-eval", tags=["code-evaluation"])

class CodeEvalRequest(BaseModel):
    answer_key: Dict[str, Any]
```

Credentials: optional `vertex_key.json` + `GOOGLE_CLOUD_PROJECT` wired before importing evaluator modules.

File mapping — backend root & API routers

Path	Role
backend/main.py	FastAPI app, CORS, lifespan (Supabase pool, OCR cleanup), mounts routers.
backend/models.py	Shared Pydantic types: AnswerKey, StudentResult, request bodies.
backend/database.py	Thin SQLite CRUD for students/submissions/results.
backend/cli.py	Standalone CLI: Drive folder → local/offline OCR → score → Sheets.
api/endpoints.py	Large router /api/*: ZIP, Drive, answer key, Sheets export, cache, pipeline runs.
api/batch_endpoints_optimized.py	High-throughput Drive/PDF batching, processing IDs, pdf-runs reconciliation.
api/code_eval_endpoints.py	/api/code-eval/*: ZIP/Drive C grading via c_code_evaluator.
smoke_test.py, test_*.py	Manual/integration probes (HEIC, phases, photos).

File mapping — c_code_evaluator/

File	Role
<code>evaluate_c_from_image.py</code>	Extract C from image (Vertex), compile, OCR syntax fix helpers.
<code>leetcode_mode.py</code>	Resolve problem slugs + run hidden tests LeetCode-style.
<code>generate_error_images_nanobanana.py</code>	Utility for error visualization / tooling (batch images).

Backend service inventory

Module	Responsibility
answer_key_service	Parse/normalize keys (Drive/ZIP/manual).
evaluation_service	Pure scoring vs SMCQ/MMCQ/NCQ rules.
batch_evaluation_service	Orchestrate match/score at batch scale.
drive_service	Drive list/download; SA vs API-key mode.
sheets_service	Read rosters; write marks, comments, per-Q columns.
drive_rename_service	Post-score file renaming on Drive.
result_cache_service	OCR + eval cache; SQLite or Supabase backend.
optimized_database_service	Pooled SQLite + batch writes.
database.py	Legacy CRUD entry points.
ocr_service	Vertex/Gemini Flash path (single-file legacy).
optimized_ocr_service	Multi-endpoint pooled OCR (Flash/Lite).
multi_region_ocr_service	Load-balanced regions/models.
image_preprocessing	Resize/compress before model payload.
pdf_split_service	PDF fan-out into per-page images.
supabase_client	Shared asyncpg pool for Supabase PG.
offline_ocr_service	Optional offline path (dev/air-gap).

Pydantic domain summary

Grading contracts (models.py) — SMCQ/MMCQ/NCQ + StudentResult aggregates.

```
1 class AnswerKeyEntry(BaseModel):
2     question_type: str
3     correct_answer: str
4     positive_marks: float = 1.0
5     negative_marks: float = 0.0
6
7 class StudentResult(BaseModel):
8     entry_number: str
9     name: str
10    total_score: float
11    max_score: float
12    details: List[QuestionResult] = []
13    file_name: Optional[str] = None
14    file_id: Optional[str] = None
```

models.py: pipeline request shapes

```
class ProcessFolderRequest(BaseModel):
    folder_url: str
    evaluation_id: Optional[str] = None
    rename_drive_inline: bool = False
    group_multiple_pages: bool = False

class PipelineSummary(BaseModel):
    model_config = {"arbitrary_types_allowed": True}
    total_students_processed: int
    answer_key_source: str
    results: List[Any] = []
    errors: List[Dict] = []
    processing_stats: Dict = {}
```

answer_key_service.py: entry point

```
1 class AnswerKeyService:
2     def extract_answer_key(self, file_path: str, mime_type: str = None) -> AnswerKey:
3         """
4         Main entry point. Detects format and routes to the right parser.
5         """
6         ...
```

Supports CSV/XLSX/PDF/images; uses Gemini when vision parsing needed (streamGenerateContent URL configured like OCRService).

batch_evaluation_service.py: async batch scoring

```
class BatchEvaluationService:
    async def process_batch_async(
        self,
        answer_key: AnswerKey,
        student_ocr_results: List[Dict],
        use_multiprocessing: bool = False
    ) -> Tuple[List[StudentResult], BatchProcessingStats]:
        optimized_key = self.optimize_answer_key(answer_key)
        if use_multiprocessing and len(student_ocr_results) > 10:
            results = await self._process_with_multiprocessing(...)
        else:
            results = await self._process_with_threading(...)
```

evaluation_service.py: pure scorer API

```
class EvaluationService:
    @staticmethod
    def match_and_score(answer_key: AnswerKey, student_answers: Dict) -> StudentResult:
        """
        Pure function - matches student answers against the answer key.
        student_answers: {
            "entry_number": "...",
            "name": "...",
            "answers": {1: "A", ...},
            "comments": "..." # optional
        }
        """
        ...
```

database.py: typed inserts

```
def add_student(self, student: Student):  
    c.execute(  
        "INSERT OR REPLACE INTO students (id, name, roll_number) VALUES (?, ?, ?)",  
        (student.id, student.name, student.roll_number),  
    )
```

IDE: collapse backend/services/ tree + open batch_endpoints_optimized.py

Backend extrapolation (future)

- **OpenAPI**: publish versioned schema; codegen clients for React + scripts.
- **Worker split**: move long PDF/ZIP jobs to Celery/RQ + object storage for inputs.
- **Idempotency keys** per (evaluation_id, file_id) on POST endpoints.
- **Policy layer**: per-organization quotas and feature flags (rename-on-Drive, Sheets export).
- **Formal audit log** table (who triggered batch, key hash, model version).

Part III

Database & persistence

Two persistence planes

SQLite `evaluation.db` Primary app store for students/submissions/results; accessed via Database and `OptimizedDatabaseService`.

Supabase Postgres Optional when `CACHE_BACKEND=supabase: result_cache, pdf_page_map; asyncpg` pool from `SUPABASE_DB_URL`.

Rationale: SQLite keeps local/dev simple; Supabase lifts cache + PDF reconciliation into a shared HA store for deployed multi-instance backends.

File mapping — backend/migrations/*.sql

SQL file	What it creates / changes
001_result_cache.sql	Table public.result_cache: OCR/eval JSONB, LRU stats, indexes; enable RLS.
002_cache_evaluation_scope.sql	Columns evaluation_id, file_id; backfill from legacy ocr:... keys; composite index.
002_pdf_page_map.sql	Table public.pdf_page_map: per-run page rows, scores, reconciliation status; RLS on.

Run statements in Supabase SQL Editor (or psql) before pointing production CACHE_BACKEND=supabase at the project.

SQLite bootstrap (database.py)

```
c.execute('''CREATE TABLE IF NOT EXISTS students
            (id TEXT PRIMARY KEY, name TEXT, roll_number TEXT)''')
c.execute('''CREATE TABLE IF NOT EXISTS submissions
            (id TEXT PRIMARY KEY, student_id TEXT, exam_id TEXT,
             file_id TEXT, status TEXT, extracted_data TEXT)''')
c.execute('''CREATE TABLE IF NOT EXISTS results
            (submission_id TEXT PRIMARY KEY, score REAL,
             feedback TEXT, details TEXT)''')
```

SQLite performance path (OptimizedDatabaseService)

WAL + mmap + bounded pool (optimized_database_service.py).

```
conn.execute("PRAGMA journal_mode=WAL")
conn.execute("PRAGMA synchronous=NORMAL")
conn.execute("PRAGMA cache_size=10000")
conn.execute("PRAGMA temp_store=MEMORY")
conn.execute("PRAGMA mmap_size=268435456")
```

Supabase result_cache (migration 001)

```
CREATE TABLE IF NOT EXISTS public.result_cache (  
  file_hash      text PRIMARY KEY,  
  file_name      text NOT NULL,  
  ocr_result     jsonb NOT NULL,  
  evaluation_result jsonb,  
  answer_key_hash text,  
  created_at     timestampz NOT NULL DEFAULT now(),  
  last_accessed  timestampz NOT NULL DEFAULT now(),  
  access_count   integer NOT NULL DEFAULT 0,  
  result_size    integer NOT NULL DEFAULT 0  
);  
ALTER TABLE public.result_cache ENABLE ROW LEVEL SECURITY;
```

Evaluation-scoped columns (migration 002)

```
ALTER TABLE public.result_cache
  ADD COLUMN IF NOT EXISTS evaluation_id text,
  ADD COLUMN IF NOT EXISTS file_id      text;

CREATE INDEX IF NOT EXISTS idx_result_cache_eval_file
  ON public.result_cache(evaluation_id, file_id);
```

Legacy ocr:... keys backfilled via split_part; modern rows use sheet:v2:... consolidated keys in application code.

pdf_page_map (PDF batch reconciliation)

```
CREATE TABLE IF NOT EXISTS public.pdf_page_map (  
    run_id          text NOT NULL,  
    pdf_hash        text NOT NULL,  
    page_number     integer NOT NULL,  
    entry_number    text,  
    name            text,  
    status          text NOT NULL,  
    total_score     numeric,  
    max_score       numeric,  
    PRIMARY KEY (run_id, page_number)  
);
```

Application cache key (result_cache_service.py)

```
def sheet_cache_key(evaluation_id: str, file_id: str) -> str:  
    return f"sheet:v2:{evaluation_id}:{file_id}"
```

One JSON row holds OCR payload + optional scored evaluation; answer_key_hash gates stale recomputation when faculty edits the key.

result_cache_service.py: backends & keys

```
"""
Supports sqlite (default) or supabase via CACHE_BACKEND.
Public API: get_cached_ocr_result, cache_ocr_result,
get_cached_evaluation_result, cache_evaluation_result, cleanup_cache, ...
"""

def sheet_cache_key(evaluation_id: str, file_id: str) -> str:
    return f"sheet:v2:{evaluation_id}:{file_id}"
```

Also: SHA-256 file hashing, answer-key hashing, LRU cleanup and purge-by-evaluation endpoints.

Supabase: `result_cache` + `pdf_page_map` with `evaluation_id` filter

Database extrapolation (future)

- Single migration tool spanning SQLite bootstrap + Supabase (Flyway/Alembic meta).
- Materialized views for analytics (latency histograms per cohort).
- Encrypted-at-rest secrets rotation doc; connection pooling limits (SUPABASE_POOL_*).
- Optional read replica or Edge cache for heavy reporting dashboards.

Part IV

OCR pipelines & vision models

OCR layers (how they differ)

OCRService Legacy single-call Gemini Flash path; service account *or* Google AI Studio API key URL; used by older `/process-file` flows.

OptimizedOCRService Peak-load design: many Vertex endpoints, aiohttp pools, image resize/JPEG quality knobs, chunk parallelism.

MultiRegionOCRService Explicit regional endpoints + **LoadBalancer** scoring (load, latency, model priority).

batch_endpoints_optimized Policy layer: **Gemini 2.5 Pro only**, dual independent OCR passes, majority tie-break third pass; full skip on cache hit.

image_preprocessing Shared downsampling/normalization feeding model payloads.

pdf_split_service Renders each PDF page to JPEG; emits synthetic Drive-shaped dicts + stable `pdf:<hash>:p<n>` IDs for cache hits on re-upload.

offline_ocr_service Optional local engines (Tesseract, Paddle, EasyOCR) matching OCR JSON shape without cloud LLM.

File mapping — OCR & ingress modules

File	Role
<code>services/ocr_service.py</code>	Legacy Gemini Flash streaming calls; base64 image payloads; used by older endpoints.
<code>services/optimized_ocr_service.py</code>	Multi-region <code>generateContent</code> ; PIL resize; aiohttp session pooling; chunk batch size tuning.
<code>services/multi_region_ocr_service.py</code>	Many region $\times$ model endpoints with health/load-aware routing.
<code>services/image_preprocessing.py</code>	<code>preprocess_for_ocr</code> : paper isolation (HSV contours), resize, JPEG bytes for API.
<code>services/pdf_split_service.py</code>	PyMuPDF render at DPI; <code>compute_pdf_hash</code> ; sheet dicts compatible with ZIP/Drive pipeline.
<code>services/offline_ocr_service.py</code>	CLI/local OCR engines; same consumer shape as cloud OCR for scoring.
<code>extract_text_from_images.py</code> (repo root)	Standalone script: folder of images $\rightarrow$ Vertex/Gemini text (dev/testing).
<code>api/batch_endpoints_optimized.py</code>	Production batch orchestration: downloads, dual OCR, cache, DB writes, pdf-runs APIs.

OCRService: Vertex URL shape (excerpt)

```
self.location = "us-central1"  
self.model_id = "gemini-2.5-flash"  
  
self.vertex_url = (  
    f"https://{self.location}-aiplatform.googleapis.com/v1/"  
    f"projects/{self.project_id}/locations/{self.location}/"  
    f"publishers/google/models/{self.model_id}:streamGenerateContent "  
)
```

OptimizedOCRService: module intent

```
"""
Optimized Multi-Region OCR Service for Peak Load
=====
Optimizations:
1. Aggressive connection pooling with keep-alive
2. Parallel batch processing with optimal chunk sizes
3. Image compression and payload optimization
4. Smart retry logic with exponential backoff
"""
```

MultiRegionOCRService: endpoint scoring

```
def score_endpoint(ep: OCREndpoint) -> float:
    load_factor = ep.current_load / ep.max_concurrent
    time_factor = ep.avg_response_time / 2.0
    priority_factor = self.model_priority.get(ep.model, 4) / 4.0
    error_rate = ep.total_errors / max(ep.total_requests, 1)
    return load_factor + time_factor + priority_factor + error_rate
```

Optimized batch: reliability banner

```
"""
Reliability guarantee:
  1. Every image is OCR'd twice independently on different Pro regions.
  2. If answer sets agree completely -> use the result immediately.
  3. If ANY answer disagrees -> run a 3rd tiebreaker pass (majority vote).

Evaluation results are FULLY CACHED - cache hits skip OCR AND evaluation entirely.
"""
```

image_preprocessing.py: public API

```
def preprocess_for_ocr(  
    image_path: str,  
    max_size: Tuple[int, int] = (1920, 1920),  
    jpeg_quality: int = 85,  
    ) -> Tuple[bytes, str]:  
    """  
    Crop paper region, resize, compress -> (jpeg_bytes, "image/jpeg")  
    """
```

pdf_split_service.py: stable PDF identity

```
1 DEFAULT_DPI = 220
2 DEFAULT_JPEG_QUALITY = 90
3
4 def compute_pdf_hash(pdf_path: str) -> str:
5     """SHA-256 of the raw PDF bytes. Stable across re-uploads."""
6     ...
```

Synthetic file IDs like pdf:<pdf_hash>p<idx> align caches across identical PDF bytes.

offline_ocr_service.py: local engines

```
"""
Local objective-sheet extraction (no cloud LLM).
Engines (CLI: --local-ocr):
- tesseract
- paddle
- easyocr
- ensemble: merge parsed answers from every installed backend
"""
```

drive_rename_service.py: naming policy

```
"""
Naming conventions:
- Matched (confident): {EntryNumber} - {Name}.{ext}
- Matched (fuzzy):      FUZZY - {EntryNumber} - {Name}.{ext}
- Answer key:          ANSWER_KEY - {original}.{ext}
"""
```

cli.py: headless pipeline

```
"""
CLI: Drive folder -> OCR (local engines or online Gemini) -> score -> Google Sheets.

Run from the backend directory:
    cd backend && python cli.py --help
"""
```

OCR extrapolation (future)

- **Model registry**: per-tenant routing (Flash for drafts, Pro for finals).
- **Confidence scores** surfaced to UI; thresholded human review queue.
- **Layout-aware OCR** (detect bubbles vs free response regions) before LLM JSON.
- Synthetic **golden-image** regression suite per exam template.
- Cost telemetry: tokens / image / cohort budget dashboards.

JSON from `/batch/ocr-debug/{processing_id}` or saved debug artifact

Part V

External services & configuration

File mapping — external integration (backend modules)

Module	External system & responsibility
<code>drive_service.py</code>	Google Drive API v3: list folder, download bytes, batch rename; SA or API key.
<code>sheets_service.py</code>	Google Sheets API: detect columns, match entry numbers, batchUpdate values.
<code>answer_key_service.py</code> / <code>ocr_service.py</code> / OCR variants	Vertex AI / Gemini HTTP (aiplatform.googleapis.com or AI Studio key URL).
<code>supabase_client.py</code>	Supabase-hosted Postgres via asyncpg pool (<code>SUPABASE_DB_URL</code>).
frontend ... <code>supabaseClient.js</code>	Supabase Auth + realtime REST (anon key) for profiles, evaluations, etc.

Google Drive (drive_service.py)

- Scope: `https://www.googleapis.com/auth/drive` (full access enables rename-after-grade flows).
- Credentials: service account JSON (`credentials.json` or `GOOGLE_APPLICATION_CREDENTIALS`); fallback `GOOGLE_API_KEY` for **public** folders only.
- Answer-key detection via regex patterns (`answer_key`, `marking_scheme`, etc.).
- Downloads via `MediaIoBaseDownload`; listing recursively from folder IDs extracted from shared URLs.

drive_service.py: scopes + credential priority

```
class DriveService:
    SCOPES = [
        'https://www.googleapis.com/auth/drive',
    ]
    # 1. Service account JSON (credentials.json or GOOGLE_APPLICATION_CREDENTIALS)
    # 2. Else GOOGLE_API_KEY for public folders only
```

Google Sheets (`sheets_service.py`)

- Scope: `spreadsheets read/write`.
- Header aliasing for entry numbers, names, total marks, comments.
- Entry normalization pattern `yyyyBBBnnnn`; OCR typo correction helpers / deduped logs for noisy polling paths.
- Optional per-question columns (`1, Q1, ...`) when exporting detailed breakdowns.

sheets_service.py: header aliases + entry regex

```
ENTRY_NUMBER_ALIASES = {  
    'entry number', 'roll number', 'enrollment', 'student id',  
}  
  
ENTRY_NUMBER_PATTERN = re.compile(  
    r'(\d{4})\s*([A-Za-z]{2,4})\s*(\d{2,5})',  
)
```

- Regional HTTPS endpoints on `aiplatform.googleapis.com`; OAuth bearer from service accounts.
- Models in deployment paths today include `gemini-2.5-flash`, `gemini-2.5-flash-lite`, `gemini-2.5-pro` depending on module.
- Separate-from-browser credential wiring for `c_code_evaluator` imports under `code-eval` routes.

Supabase (dual role)

- **Auth + profiles:** frontend @supabase/supabase-js; row-level access patterns live in Supabase SQL for user tables (not shown in backend Python).
- **Operational Postgres:** backend SUPABASE_DB_URL + asyncpg pool (supabase_client.py); toggled by CACHE_BACKEND=supabase.
- Pool sizing env: SUPABASE_POOL_MIN, SUPABASE_POOL_MAX; statement_cache_size=0 for PgBouncer compatibility.

Supabase pool bootstrap (backend)

```
1 _pool = await asyncpg.create_pool(  
2     dsn=_get_dsn(),  
3     min_size=int(os.getenv("SUPABASE_POOL_MIN", "1")),  
4     max_size=int(os.getenv("SUPABASE_POOL_MAX", "10")),  
5     command_timeout=30,  
6     statement_cache_size=0,  
7 )
```

Environment variables (representative)

Variable	Purpose
VITE_BACKEND_URL	Frontend → FastAPI base URL.
VITE_SUPABASE_URL, VITE_SUPABASE_ANON_KEY	Browser auth session.
GOOGLE_APPLICATION_CREDENTIALS	SA JSON for Drive/Sheets/Vertex.
GOOGLE_API_KEY / VERTEX_AI_API_KEY	Fallback Studio/API access paths.
GOOGLE_CLOUD_PROJECT	Vertex project id resolution.
CACHE_BACKEND	sqlite (default) vs supabase.
SUPABASE_DB_URL	Postgres DSN for cache/pdf tables.
CORS_ORIGINS	Allowed browser origins for credentialed CORS.

GCP Console: enabled APIs + service account keys redacted

External-services extrapolation (future)

- **Workload Identity** instead of long-lived JSON keys on servers.
- **Dedicated Shared Drive** ACL templates documented per institution.
- **Secrets manager** integration (Vault / GSM) for rotating SA keys.
- Regional **egress controls** if student data must stay in-country.

Cross-cutting future themes

- Observability: trace IDs across ZIP/PDF runs; OCR disagreement metrics.
- Fairness: blind grading mode (strip names earlier in pipeline).
- Extensibility: plugin scorer for new question types; LMS CSV adapters beyond Sheets.
- Compliance: data retention windows on cache tables + purge automation.

Questions & milestones

Build: `pdflatex presentation3_future_development.tex`